Indraprastha Institute of Information Technology Delhi (IIITD)

Department of Electronics and Communication Engineering

Triggering Based Real-Time Radar Signal Processing on Edge

Course: AELD – Project(1)

Submitted to: Professor Dr. Sumit J. Darak

Submitted by Group 5:

Anant Gyan Singhal (2023082)
Kinshuk
Garima
Abhishek

Advisor: Dr. Shobha Sundar Ram & Dr. Sumit Darak

Date: 26 January 2025

IIITD, New Delhi, India

Contents

1 Project Details **3**
1.1 Project Objectives . . . 3
1.2 Project Description . . . 3
1.3 Project Specification . . . 3

2 Theoretical Framework of Radar Systems **3**
2.1 Fundamental Radar Equations . . . 4
2.1.1 Wavelength . . . 4
2.1.2 Range Estimation . . . 4
2.1.3 Range Resolution and Range Axis . . . 4
2.1.4 Velocity, Doppler Shift, and Doppler Phase . . . 4
2.1.5 Pulse Repetition Interval, PRF, and Doppler Resolution . . . 5
2.1.6 AWGN and SNR . . . 5

3 Theoretical Background of Radar Waveforms **5**
3.1 Golay Complementary Sequences . . . 5
3.1.1 Aperiodic Autocorrelation . . . 6
3.1.2 Recursive Generation . . . 6
3.2 Zadoff-Chu (ZC) Sequences . . . 6

4 Hardware and Block Design Architecture **7**
4.1 Transmitter Block . . . 7
4.2 Receiver Block . . . 7
4.3 Trigger IP Logic . . . 7

5 Methodology **7**
5.1 Golay Sequence Based Single Carrier Radar . . . 7
5.1.1 Sequence Generation and Transmit Construction . . . 7
5.1.2 Channel Modeling . . . 7
5.1.3 Range Processing . . . 8
5.1.4 Data Cube and Doppler Processing . . . 8
5.2 Zadoff-Chu (ZC) Sequence Based Multi-Carrier Radar . . . 8
5.2.1 Sequence Generation and Transmit Construction . . . 8
5.2.2 Range Processing in the Frequency Domain . . . 8
5.2.3 Doppler Processing . . . 8

6 C Code Implementation: Unoptimized and Optimized **9**
6.1 Zadoff-Chu Radar – Unoptimized . . . 9
6.1.1 ZC-U Part 1: Defines, Constants, and Global Memory . . . 9
6.1.2 ZC-U Part 2: Gaussian RNG . . . 9
6.1.3 ZC-U Part 3: FFT and IFFT . . . 10
6.1.4 ZC-U Part 4: ZC Sequence Generation and Transmit Buffer . . . 11
6.1.5 ZC-U Part 5: Channel Simulation . . . 11
6.1.6 ZC-U Part 6: Range Compression . . . 12
6.1.7 ZC-U Part 7: Doppler Processing and Peak Detection . . . 12
6.1.8 ZC-U Part 8: RMSE Computation . . . 13

6.2 Zadoff-Chu Radar – Optimized . . . 13
6.2.1 ZC-O Part 1: New Defines and Memory . . . 13
6.2.2 ZC-O Part 2: Twiddle Table Initialization . . . 14
6.2.3 ZC-O Part 3: Optimized FFT . . . 14
6.2.4 ZC-O Part 4: Precomputed Conjugate Reference . . . 15
6.2.5 ZC-O Part 5: AWGN Optimisation . . . 15
6.2.6 ZC-O Part 6: Execution Time Measurement . . . 16
6.3 Golay Radar – Unoptimized . . . 16
6.3.1 G-U Part 1: Defines, Constants, and Global Memory . . . 16
6.3.2 G-U Part 2: Gaussian RNG and Signal Power . . . 17
6.3.3 G-U Part 3: FFT and IFFT . . . 17
6.3.4 G-U Part 4: Golay Sequence Generation . . . 18
6.3.5 G-U Part 5: Transmit Buffer Setup and FFT Pre-computation . . . 18
6.3.6 G-U Part 6: Channel Simulation . . . 19
6.3.7 G-U Part 7: Range Compression . . . 19
6.3.8 G-U Part 8: Doppler Processing and Peak Detection . . . 20
6.3.9 G-U Part 9: RMSE and Timing . . . 21
6.4 Golay Radar – Optimized . . . 21
6.4.1 G-O Part 1: New Defines and Memory . . . 21
6.4.2 G-O Part 2: Twiddle Table and Optimized FFT . . . 22
6.4.3 G-O Part 3: Precomputed Conjugate References . . . 22
6.4.4 G-O Part 4: Optimized Correlation . . . 23
6.4.5 G-O Part 5: AWGN Optimisation . . . 23
6.4.6 G-O Part 6: Doppler FFT – Direct In-Place on Cube . . . 24
6.4.7 G-O Part 7: Peak Detection – Squared Magnitude . . . 24

7 Performance Evaluation and Outputs **24**
7.1 Monte Carlo Simulation Framework . . . 24
7.2 RMSE Metrics . . . 25
7.3 Golay Radar Performance . . . 25
7.4 ZC Radar Performance . . . 25

1 Project Details

1.1 Project Objectives

The primary objectives of this project are:

i. Implement a complete Golay Sequence based radar on the Processing System (PS) featuring Trigger Logic.

ii. Implement a complete Zadoff-Chu (ZC) Sequence based radar on the PS featuring Trigger Logic.

iii. Calculate Range and Doppler Root Mean Square Error (RMSE) for both radar implementations across varying Signal-to-Noise Ratio (SNR) conditions.

1.2 Project Description

This project focuses on developing *Triggering Based Real-Time Radar Signal Processing on the Edge*, specifically targeting the rapidly expanding Automotive Radar Market. The global automotive radar market is projected to grow from USD 7.2 Billion in 2025 to USD 29.2 Billion by 2035 at a CAGR of 15%, underscoring the urgency and relevance of edge-deployable radar solutions.

Radar (Radio Detection and Ranging) systems operate by emitting electromagnetic waves and processing reflected signals to detect targets. Unlike optical systems, radar operates continuously (24×7) and can penetrate environmental obstacles such as clouds, fog, walls, and the ground. The system architecture transmits specific signal sequences (Golay or ZC) through a simulated physical channel, which introduces propagation delay, Doppler frequency shifts, and Additive White Gaussian Noise (AWGN). The receiver then processes these signals to extract accurate target range and velocity data.

1.3 Project Specification

The edge radar system is configured with the parameters shown in Table 1.

Table 1: System Specifications

Parameter	**Symbol**	**Value**
Centre Frequency	F_c	30 GHz
Bandwidth	β	300 MHz
Range Resolution	Δr	0.5 m
Maximum Range	$R_{\max}$	256 m
Velocity Resolution	Δv	2.86 m/s
Maximum Velocity	$v_{\max}$	±732.42 m/s

2 Theoretical Framework of Radar Systems

To accurately process signals at the edge, the system relies on fundamental physics and mathematical principles.

2.1 Fundamental Radar Equations

2.1.1 Wavelength

The carrier wavelength λ links the speed of light to the operating frequency and governs both the Doppler effect and the antenna aperture requirements:

$$\lambda = \frac{c}{f_c} = \frac{3 \times 10^8}{30 \times 10^9} = 0.01 \text{ m} \tag{1}$$

2.1.2 Range Estimation

In a monostatic radar setup (where the transmitter and receiver are collocated), the distance to a target R is proportional to the round-trip time delay T_d of the reflected pulse:

$$R = \frac{cT_d}{2} \tag{2}$$

where $c = 3 \times 10^8$ m/s is the speed of light. As a target moves at constant radial velocity v_{true}, its instantaneous range evolves continuously over slow-time $t_{\text{now}} = p \cdot \text{PRI}$ (where p is the pulse index):

$$R_{\text{now}} = R_{\text{true}} + v_{\text{true}} \cdot t_{\text{now}} \tag{3}$$

The corresponding round-trip propagation delay is:

$$\tau = \frac{2R_{\text{now}}}{c} \tag{4}$$

In the discrete implementation this delay is realised as an integer sample offset $d = \lfloor \tau f_s \rfloor$.

2.1.3 Range Resolution and Range Axis

The ability to distinguish two closely spaced targets (range resolution Δr) is inversely proportional to the signal bandwidth β. Since the sampling frequency f_s sets the effective bandwidth in the discrete system:

$$\Delta r = \frac{c}{2\beta} = \frac{c}{2f_s} = \frac{3 \times 10^8}{2 \times 300 \times 10^6} = 0.5 \text{ m/bin} \tag{5}$$

Each row of the radar data cube therefore represents exactly 0.5 m of target distance. The range axis array is constructed by scaling each bin index by Δr.

2.1.4 Velocity, Doppler Shift, and Doppler Phase

When a target moves with radial velocity v, it induces a Doppler frequency shift f_d on the carrier:

$$f_d = \frac{2v}{\lambda} \tag{6}$$

Rearranging to convert measured Doppler frequency bins back to physical velocity:

$$v = \frac{f_d \, \lambda}{2} \tag{7}$$

In the time domain, this Doppler shift manifests as a complex phase rotation applied to each received pulse:

$$s_{\text{Doppler}}(t) = s_{\text{delayed}}(t) \cdot e^{j2\pi f_d \, t_{\text{now}}} \tag{8}$$

2.1.5 Pulse Repetition Interval, PRF, and Doppler Resolution

The Pulse Repetition Interval (PRI) is the total time window occupied by one padded pulse of length L samples:

$$\text{PRI} = \frac{L}{f_s} \tag{9}$$

The Pulse Repetition Frequency (PRF) is its reciprocal:

$$\text{PRF} = \frac{1}{\text{PRI}} = \frac{f_s}{L} \tag{10}$$

For both the Golay and ZC radars, $L = 2N = 1024$, giving $\text{PRI} = 1024/(300 \times 10^6) \approx 3.41\ \mu\text{s}$. The PRI governs the maximum unambiguous range, while the PRF governs the unambiguous velocity extent.

The Doppler frequency resolution is set by the total coherent dwell time $T_{\text{dwell}} = N_p \cdot \text{PRI}$, where N_p is the number of processed pulses:

$$\Delta f_d = \frac{1}{T_{\text{dwell}}} = \frac{\text{PRF}}{N_p} \tag{11}$$

2.1.6 AWGN and SNR

Additive White Gaussian Noise (AWGN) is injected at the receiver to model thermal and atmospheric interference. The noise standard deviation is derived from the desired SNR and the instantaneous signal power P_s:

$$P_n = \frac{P_s}{\text{SNR}_{\text{linear}}}, \qquad \sigma = \sqrt{\frac{P_n}{2}} \tag{12}$$

Complex noise samples are generated as $n[k] = \sigma(\mathcal{N}(0,1) + j\,\mathcal{N}(0,1))$ and added to the received signal.

3 Theoretical Background of Radar Waveforms

3.1 Golay Complementary Sequences

Golay complementary sequences are highly advantageous in modern radar signal processing. A Golay pair consists of two distinct, equal-length binary sequences, A and B, defined as:

$$A = (a_0,\ a_1,\ \ldots,\ a_{N-1}) \tag{13}$$

$$B = (b_0,\ b_1,\ \ldots,\ b_{N-1}) \tag{14}$$

where each element takes values $a_i, b_i \in \{+1, -1\}$.

3.1.1 Aperiodic Autocorrelation

For a sequence x, the aperiodic autocorrelation at lag k is defined as:

$$R_x(k) = \sum_{j=0}^{N-k-1} x_j \, x_{j+k} \tag{15}$$

The defining property of a Golay pair is that when the autocorrelation functions of Sequence A and Sequence B are independently computed and algebraically summed, the result is an ideal impulse:

$$R_A(k) + R_B(k) = \begin{cases} 2N & \text{for } k = 0 \\ 0 & \text{for } k \neq 0 \end{cases} \tag{16}$$

This property translates directly to **complete sidelobe cancellation** in radar engineering. Standard radar pulses yield unwanted sidelobes that can mask smaller targets. By transmitting Sequence A followed by Sequence B, correlating each at the receiver, and summing the results, all out-of-phase sidelobes destructively interfere.

Furthermore, the combined power spectrum of sequences A and B is entirely flat:

$$S_A(f) + S_B(f) = C \quad \text{(constant)} \tag{17}$$

3.1.2 Recursive Generation

Sequences are generated recursively starting from the initial pair $A_1 = [1,\ 1]$ and $B_1 = [1,\ -1]$. At each iteration n, the sequence length is doubled using the recurrences:

$$A_n = [A_{n-1},\ \ B_{n-1}] \tag{18}$$
$$B_n = [A_{n-1},\ -B_{n-1}] \tag{19}$$

3.2 Zadoff-Chu (ZC) Sequences

Zadoff-Chu sequences are complex-valued sequences belonging to the CAZAC (Constant Amplitude Zero AutoCorrelation) family. A ZC sequence's phase varies quadratically with the sequence index n:

$$x(n) = e^{-j\pi n^2/N}, \quad n = 0, 1, \ldots, N-1 \tag{20}$$

Incorporating the root index u, the general formulation becomes:

$$x_u[n] = e^{-j\pi u n^2/N_{\text{sample}}} \tag{21}$$

Because every element lies on the unit circle in the complex plane ($|x(n)| = 1$), the sequence maintains a constant amplitude. This property allows the waveform to pass through non-linear RF power amplifiers without distortion, preserving signal power efficiency.

Additionally, a ZC sequence correlated with a cyclically shifted version of itself produces a sharp peak at the correct alignment and negligible correlation elsewhere. Unlike Golay sequences, which require a complementary pair, a *single* ZC sequence provides robust correlation performance.

4 Hardware and Block Design Architecture

The hardware architecture relies on a Zynq Processing System (PS) designed for parallel real-time processing of radar signals.

4.1 Transmitter Block

Data flows from the Zynq PS into a Tx DMA and a Tx FIFO. This simulated transmit signal passes through a channel model that applies delay, Doppler frequency shifts, and AWGN.

4.2 Receiver Block

The signal is captured by the receiver, which features a custom Trigger IP and an Rx DMA. Once triggered, the data undergoes correlation and FFT processing to produce a 2D Range-Doppler Map.

4.3 Trigger IP Logic

Real-time continuous processing is managed by the Trigger IP. The Zynq PS sends data batches of 32,768 samples (the maximum DMA transfer limit) to DDR memory. The Trigger IP captures these 32,768 samples 16 times, waiting until a full block of 2,048 samples (16×16) is assembled before asserting the Rx DMA to pass data to the Radar Signal Processor (RSP). Upon transmitting the final sample, the Trigger IP asserts a low signal to reset the pipeline.

5 Methodology

5.1 Golay Sequence Based Single Carrier Radar

5.1.1 Sequence Generation and Transmit Construction

The transmitter recursively generates a 512-sample Golay pair (A, B) using the recurrences in Equations (8) and (9). To prevent circular convolution artefacts during FFT-based receiver processing, a block of zero-padding samples is appended to each sequence. The system alternates the transmission of Sequence A and Sequence B on successive pulses, and the system transmits a 2048×256 matrix block in total.

5.1.2 Channel Modeling

The transmitted signal propagates through a channel that applies:

- A **propagation delay** $\tau = 2R/c$, applied as an integer sample offset $d = \lfloor \tau F_s \rfloor$,
- A **Doppler phase rotation** $e^{j2\pi f_d t}$ applied sample-wise, and
- **AWGN** scaled to the desired SNR level.

5.1.3 Range Processing

Range estimation is performed using FFT-based matched filtering. For a received signal $r[n]$ and a known transmit signal $t[n]$, the range profile is obtained as:

$$\text{Range Profile} = \mathcal{F}^{-1}\{\mathcal{F}\{r[n]\} \cdot \mathcal{F}^{*}\{t[n]\}\} \tag{22}$$

Applying this to both complementary sequences and summing the results cancels all sidelobes:

$$P_{\text{total}}[n] = P_A[n] + P_B[n] \tag{23}$$

where P_A and P_B are the individual range profiles from sequences A and B, respectively. The output is sliced to 1024×1 and stored iteratively to form a 1024×256 matrix.

5.1.4 Data Cube and Doppler Processing

The 1D range profiles for each pulse are stored as columns in a 2D Radar Data Cube of dimension $N_r \times N_p$, where N_r is the number of range bins and N_p is the number of pulses. Doppler processing is performed by applying a 256-point FFT across the pulse dimension (rows) of the cube. This yields a Range-Doppler map, the peak of which gives the estimated target range and velocity.

5.2 Zadoff-Chu (ZC) Sequence Based Multi-Carrier Radar

5.2.1 Sequence Generation and Transmit Construction

A 512-sample complex-valued ZC sequence is generated per Equation (10). To exploit its periodic autocorrelation property in a physical linear-channel environment, a Cyclic Prefix (CP) is prepended by copying the base sequence, making the linear propagation delay appear as a perfect circular shift at the receiver:

$$\tilde{x}[n] = x[(n \bmod N)], \quad n = 0, 1, \ldots, 2N-1 \tag{24}$$

5.2.2 Range Processing in the Frequency Domain

Upon receiving the echo, the receiver removes the Cyclic Prefix to isolate the core N-sample window $r_{\text{cp}}[n]$. A 512-point FFT is applied, and the result is element-wise multiplied by the complex conjugate of the transmitted ZC sequence's FFT:

$$Y[k] = \mathcal{F}\{r_{\text{cp}}[n]\} \cdot \mathcal{F}^{*}\{x[n]\} \tag{25}$$

A 512-point IFFT then yields the time-domain correlated delay output (range profile):

$$y[n] = \mathcal{F}^{-1}\{Y[k]\} \tag{26}$$

5.2.3 Doppler Processing

The 512-point IFFT outputs are stored iteratively to form a 512×256 radar data cube. The matrix is transposed, and a 256-point 1D FFT is applied across the slow-time dimension to extract Doppler phase shifts, yielding the final 512×256 2D Delay-Doppler map.

6 C Code Implementation: Unoptimized and Optimized

This section walks through each functional block of the C implementation snippet by snippet. The Zadoff-Chu radar is presented first in full (unoptimized, then optimized), followed by the Golay radar in full (unoptimized, then optimized). All code runs on the Zynq Processing System (ARM Cortex-A9) of the Xilinx Zedboard.

6.1 Zadoff-Chu Radar – Unoptimized

6.1.1 ZC-U Part 1: Defines, Constants, and Global Memory

```c
#define NSAMP   512
#define NPULSES 512
#define FS      300e6
#define FC      30e9
#define C       3e8
#define PI      3.141592653589793
#define NUM_MC  50
#define NUM_SNR 3     // Only 3 SNR points tested

#define FIFO_SIZE_SAMPLES 32768
#define NUM_BATCHES       16
#define SAMPLES_PER_PULSE (2*NSAMP)
#define PULSES_PER_BATCH  (FIFO_SIZE_SAMPLES / SAMPLES_PER_PULSE)

typedef double complex cfloat;

cfloat zc[NSAMP];           // Base ZC sequence
cfloat tx[2*NSAMP];         // Cyclic-prefix transmit buffer
cfloat rx[2*NSAMP];         // Received signal
cfloat rx_win[NSAMP];       // CP-stripped receive window
cfloat range_fft[NSAMP];    // Working FFT buffer
cfloat zc_fft[NSAMP];       // FFT of reference ZC (used per pulse)
cfloat cube[NSAMP][NPULSES];
cfloat doppler_vec[NPULSES];

double snr_db_list[NUM_SNR] =
   {-45,-44,-43,-42,-41,-40,-39,-38,-37,-36,-35,-34,-33,-32,-31,-30};
```

Listing 1: ZC unoptimized – defines and global memory

The unoptimized ZC version no precomputed conjugate array for the reference FFT. `FIFO_SIZE_SAMPLES = 32768` and `NUM_BATCHES = 16` map directly to the Trigger IP hardware constraint: each DMA burst transfers at most 32,768 samples. With each pulse being $2N = 1024$ samples, one batch holds 32 pulses, and 16 batches deliver all 512 required pulses.

6.1.2 ZC-U Part 2: Gaussian RNG

```c
double randn()
{
    static int haveSpare = 0;
    static double spare;

    if (haveSpare) { haveSpare = 0; return spare; }
```

```
    haveSpare = 1;

    double u, v, s;
    do {
        u = (double)rand() / RAND_MAX * 2 - 1;
        v = (double)rand() / RAND_MAX * 2 - 1;
        s = u*u + v*v;
    } while (s >= 1 || s == 0);

    s = sqrt(-2 * log(s) / s);
    spare = v * s;
    return u * s;
}
```

Listing 2: ZC unoptimized – Box-Muller RNG

This is the polar form of the Box-Muller transform. Two uniform samples are drawn from $[-1, 1]$ and rejected if they fall outside the unit circle. The scale $\sqrt{-2 \ln s/s}$ then maps them to two independent standard-normal samples. One is returned immediately; the other is cached in `spare` and returned on the next call, so only one `sqrt` and `log` pair is needed for every two noise samples.

6.1.3 ZC-U Part 3: FFT and IFFT

```
void fft512(cfloat *x)
{
    int i, j = 0, stages = 9;

    // Bit-reversal permutation
    for (i = 1; i < NSAMP; i++) {
        int bit = NSAMP >> 1;
        while (j & bit) { j ^= bit; bit >>= 1; }
        j |= bit;
        if (i < j) { cfloat tmp = x[i]; x[i] = x[j]; x[j] = tmp; }
    }

    // Butterfly stages
    for (i = 1; i <= stages; i++) {
        int m = 1 << i, m2 = m >> 1;
        cfloat wm = cexp(-I * 2 * PI / m);    // twiddle base:
            recomputed EVERY call

        for (int k = 0; k < NSAMP; k += m) {
            cfloat w = 1;
            for (j = 0; j < m2; j++) {
                cfloat t = w * x[k+j+m2];
                cfloat u = x[k+j];
                x[k+j]    = u + t;
                x[k+j+m2] = u - t;
                w *= wm;
            }
        }
    }
}

void ifft512(cfloat *x)
{
```

```
    for (int i = 0; i < NSAMP; i++) x[i] = conj(x[i]);
    fft512(x);
    for (int i = 0; i < NSAMP; i++) x[i] = conj(x[i]) / NSAMP;
}
```

Listing 3: ZC unoptimized – Cooley-Tukey FFT (twiddle computed per call)

The FFT implements radix-2 decimation-in-time. The bit-reversal pass reorders samples so butterfly operations proceed in-place. At each of the 9 stages the base twiddle `wm = cexp(-I*2*PI/m)` is recomputed from scratch with a `cexp()` call. On an ARM Cortex-A9 without hardware floating-point trigonometry, each `cexp()` is expensive. Since the function is called for every pulse of every Monte Carlo trial, this is the dominant runtime bottleneck.

The IFFT uses the conjugate-symmetry identity: $\mathrm{IFFT}(X) = \frac{1}{N}\overline{\mathrm{FFT}(\bar{X})}$. The input is conjugated, the forward FFT is run, and the output is conjugated and scaled, reusing the same code path without a separate inverse algorithm.

6.1.4 ZC-U Part 4: ZC Sequence Generation and Transmit Buffer

```
void generate_zc()
{
    for (int n = 0; n < NSAMP; n++) {
        double phase = -PI * n * n / NSAMP;    // quadratic phase
        zc[n] = cexp(I * phase);                // unit-magnitude sample
    }
}

// In main():
generate_zc();
for (int i = 0; i < NSAMP; i++) {
    zc_fft[i]      = zc[i];    // reference copy for correlation
    tx[i]          = zc[i];    // first half: base ZC
    tx[i + NSAMP]  = zc[i];    // second half: cyclic prefix (repeat)
}
fft512(zc_fft);                 // pre-compute reference FFT once
```

Listing 4: ZC unoptimized – sequence generation and CP transmit buffer

Each element $x[n] = e^{-j\pi n^2/N}$ lies on the complex unit circle, giving the sequence its CAZAC (Constant Amplitude Zero AutoCorrelation) property. The transmit buffer `tx[2*NSAMP]` is formed by repeating the base sequence: the first 512 samples are the base and the second 512 samples are an identical copy acting as a cyclic prefix. When the channel imposes a linear delay, reading only the second half of the received signal isolates a window where that delay appears as a circular shift — the prerequisite for valid frequency-domain circular correlation.

6.1.5 ZC-U Part 5: Channel Simulation

```
double t_now = pulse * PRI;
double r_now = true_r + true_v * t_now;    // instantaneous range
double tau   = 2 * r_now / C;              // round-trip delay
int delay_samples = (int)(tau * FS);       // convert to sample offset

cfloat doppler_phase = cexp(I * 2*PI*fd*t_now);  // Doppler phasor
```

```

for (int i = 0; i < 2*NSAMP; i++) {
    int idx = i - delay_samples;
    cfloat signal = 0;
    if (idx >= 0 && idx < 2*NSAMP)
        signal = tx[idx] * doppler_phase;          // delay + Doppler

    cfloat noise = noise_std * (randn() + I*randn()) / sqrt(2);  //
        AWGN
    rx[i] = signal + noise;
}
```

Listing 5: ZC unoptimized – channel: delay, Doppler, AWGN

The delay and Doppler are applied together in one loop. The index shift `idx = i - delay_samples` implements the propagation delay: when `idx` is negative the signal is zero (echo has not yet arrived). The Doppler shift is applied as a scalar multiplication by $e^{j2\pi f_d t_{\text{now}}}$. AWGN is added as complex Gaussian noise scaled by `noise_std / sqrt(2)`, where the $\sqrt{2}$ splits noise power equally between real and imaginary parts.

6.1.6 ZC-U Part 6: Range Compression

```
// Strip cyclic prefix: take samples NSAMP to 2*NSAMP-1
for (int i = 0; i < NSAMP; i++) rx_win[i] = rx[i + NSAMP];

// Copy window and FFT
for (int i = 0; i < NSAMP; i++) range_fft[i] = rx_win[i];
fft512(range_fft);

// Element-wise multiply by conjugate of reference -- conj() called per
    sample
for (int i = 0; i < NSAMP; i++)
    range_fft[i] *= conj(zc_fft[i]);   // runtime conjugate computation

// IFFT -> time-domain range (delay) profile
ifft512(range_fft);

// Store range profile into data cube column
for (int r = 0; r < NSAMP; r++) cube[r][pulse] = range_fft[r];
```

Listing 6: ZC unoptimized – CP removal and frequency-domain correlation

The receiver first discards the cyclic prefix by taking `rx[NSAMP]` to `rx[2*NSAMP-1]`. A 512-point FFT converts this window to the frequency domain. Correlation is then performed by element-wise multiplying with the complex conjugate of the reference FFT, implementing $Y[k] = R[k] \cdot \overline{X_{\text{ZC}}[k]}$. An IFFT converts the result back to the time domain, producing the range (delay) profile for this pulse. In the unoptimized version `conj(zc_fft[i])` is recomputed on every iteration of every pulse.

6.1.7 ZC-U Part 7: Doppler Processing and Peak Detection

```
// Doppler FFT: for each range bin, FFT over all 512 pulses
for (int r = 0; r < NSAMP; r++) {
    for (int p = 0; p < NPULSES; p++)
        doppler_vec[p] = cube[r][p];   // copy into temp buffer
```

```

    fft512(doppler_vec);
    fftshift512(doppler_vec);          // centre zero-velocity

    for (int p = 0; p < NPULSES; p++)
        cube[r][p] = doppler_vec[p];   // write back
}

// Peak detection: scan entire 2D map for maximum magnitude
double max_val = 0;
int r_idx = 0, v_idx = 0;
for (int r = 0; r < NSAMP; r++)
    for (int p = 0; p < NPULSES; p++) {
        double mag = cabs(cube[r][p]);   // calls sqrt() every cell
        if (mag > max_val) { max_val = mag; r_idx = r; v_idx = p; }
    }

// Convert bin indices to physical range and velocity
double range_est = (r_idx * C) / (2 * FS);
double fd_est    = ((double)v_idx - NPULSES/2) * (PRF / NPULSES);
double vel_est   = (fd_est * lambda) / 2;
```

Listing 7: ZC unoptimized – Doppler FFT, fftshift, and peak detection

Each row of the data cube is a slow-time sequence of 512 complex values at a fixed range bin. A 512-point FFT across this sequence extracts the Doppler phase shift accumulated over the coherent processing interval, yielding a column of the 2D Range-Doppler map. `fftshift` swaps the two halves so that zero velocity lies at the map centre. Peak detection then scans all $512 \times 512 = 262{,}144$ cells using `cabs()` which internally calls `sqrt()`, adding significant overhead. The bin index is converted to range by $\hat{R} = r_{\text{idx}} \cdot c/(2f_s)$ and to velocity via the inverse Doppler formula.

6.1.8 ZC-U Part 8: RMSE Computation

```
err_r += (range_est - true_r) * (range_est - true_r);
err_v += (vel_est   - true_v) * (vel_est   - true_v);

// After NUM_MC runs:
double rmse_r = sqrt(err_r / NUM_MC);
double rmse_v = sqrt(err_v / NUM_MC);

xil_printf("SNR = %d | RMSE Range x100 = %d | RMSE Velocity x100 = %d\n
    \r",
            (int)snr_db, (int)(rmse_r*100), (int)(rmse_v*100));
```

Listing 8: ZC unoptimized – RMSE accumulation and output

Squared estimation errors are accumulated over all 50 Monte Carlo runs. After the loop, a single `sqrt()` yields the RMSE for that SNR point. Results are scaled by 100 before printing because `xil_printf` on the Zynq PS does not support floating-point format specifiers, so integer printing of the $\times 100$ value preserves two decimal places.

6.2 Zadoff-Chu Radar – Optimized

6.2.1 ZC-O Part 1: New Defines and Memory

```c
#define NUM_SNR 16          // Extended to full 16-point SNR sweep
#define INV_SQRT2 0.70710678  // Compile-time constant replaces sqrt(2)

/* New array: conjugate of reference FFT stored once at startup */
cfloat zc_fft_conj[NSAMP];

/* Per-stage twiddle table (9 entries for 512-point FFT) */
cfloat wm_table[10];

double snr_db_list[NUM_SNR] =
    {-45,-44,-43,-42,-41,-40,-39,-38,-37,-36,-35,-34,-33,-32,-31,-30};
```

Listing 9: ZC optimized – new defines and precomputed memory

The optimized version expands the SNR sweep from 3 to 16 points, covering −45 to −30 dB. Two new data structures are added: `zc_fft_conj[]` stores the element-wise conjugate of the reference ZC FFT, computed once at startup and reused for every pulse of every Monte Carlo trial. `wm_table[]` stores the 9 per-stage base twiddle factors $e^{-j2\pi/m}$ for the FFT butterfly, eliminating all `cexp()` calls from the FFT inner loop.

6.2.2 ZC-O Part 2: Twiddle Table Initialization

```c
void init_twiddle()
{
    for (int i = 1; i <= 9; i++) {
        int m = 1 << i;                      // stage size: 2,4,8,...,512
        wm_table[i] = cexp(-I * 2*PI / m);   // one cexp() per stage total
    }
}
```

Listing 10: ZC optimized – twiddle table init (called once)

This function is called exactly once in `main()` before any processing. It computes one `cexp()` per FFT stage (9 calls total). In the unoptimized version, this same `cexp(-I*2*PI/m)` was evaluated on every invocation of `fft512()` — that is, $512 \times 50 \times 16 = 409{,}600$ times across a full run. The table reduces this to 9 calls total.

6.2.3 ZC-O Part 3: Optimized FFT

```c
void fft512(cfloat *x)
{
    int i, j = 0, stages = 9;

    // Bit-reversal (unchanged from unoptimized)
    for (i = 1; i < NSAMP; i++) {
        int bit = NSAMP >> 1;
        while (j & bit) { j ^= bit; bit >>= 1; }
        j |= bit;
        if (i < j) { cfloat tmp = x[i]; x[i] = x[j]; x[j] = tmp; }
    }

    // Butterfly stages: twiddle read from table, no cexp()
    for (i = 1; i <= stages; i++) {
```

```
        int m = 1 << i, m2 = m >> 1;
        cfloat wm = wm_table[i];   // TABLE LOOKUP -- replaces cexp()

        for (int k = 0; k < NSAMP; k += m) {
            cfloat w = 1;
            for (j = 0; j < m2; j++) {
                cfloat t = w * x[k+j+m2];
                cfloat u = x[k+j];
                x[k+j]     = u + t;
                x[k+j+m2] = u - t;
                w *= wm;
            }
        }
    }
}
```

Listing 11: ZC optimized – FFT using wm_table lookup

The only change versus the unoptimized FFT is the single line `cfloat wm = wm_table[i]` replacing `cfloat wm = cexp(-I*2*PI/m)`. The butterfly arithmetic is identical. Because the twiddle base is now a memory load rather than a transcendental function evaluation, the per-call cost of the FFT is reduced to purely arithmetic operations.

6.2.4 ZC-O Part 4: Precomputed Conjugate Reference

```
// In main(), after fft512(zc_fft):
for (int i = 0; i < NSAMP; i++)
    zc_fft_conj[i] = conj(zc_fft[i]);  // computed ONCE

// In the per-pulse correlation loop (replacing conj(zc_fft[i])):
for (int i = 0; i < NSAMP; i++)
    range_fft[i] *= zc_fft_conj[i];    // direct load, no function call
```

Listing 12: ZC optimized – precompute conjugate of reference FFT

After `fft512(zc_fft)`, one additional loop stores its conjugate in `zc_fft_conj[]`. Every subsequent pulse then multiplies by this precomputed array rather than calling `conj(zc_fft[i])` 512 times. Over $512 \times 50 \times 16 = 409{,}600$ pulses, this removes over 210 million `conj()` calls.

6.2.5 ZC-O Part 5: AWGN Optimisation

```
/* Unoptimized: */
cfloat noise = noise_std * (randn() + I*randn()) / sqrt(2);

/* Optimized: */
cfloat noise = noise_std * (randn() + I*randn()) * INV_SQRT2;
```

Listing 13: ZC optimized – INV_SQRT2 replaces runtime division

Replacing the runtime `/ sqrt(2)` with a multiply by the compile-time constant `INV_SQRT2 = 0.70710678` avoids both the `sqrt()` call and the floating-point division. Division is significantly slower than multiplication on most FPUs; this change is applied to every noise sample in the inner channel loop.

6.2.6 ZC-O Part 6: Execution Time Measurement

```c
#include "xtime_l.h"

XTime time_PS_start, time_PS_end;
XTime_SetTime(0);
XTime_GetTime(&time_PS_start);

// ... full SNR / MC / pulse loop ...

XTime_GetTime(&time_PS_end);
float time_processor =
    (float)(time_PS_end - time_PS_start) / COUNTS_PER_SECOND;

printf("Execution Time for PS in seconds: %f\n\r", time_processor);
```

Listing 14: ZC optimized – XTime profiling added

`XTime_GetTime()` reads the 64-bit global timer of the ARM Cortex-A9. `COUNTS_PER_SECOND` is a Xilinx-defined macro equal to the CPU clock (666 MHz on the Zedboard), so dividing the tick difference yields wall-clock seconds. This instrumentation is absent from the unoptimized version and is used to quantify the total speedup achieved.

6.3 Golay Radar – Unoptimized

6.3.1 G-U Part 1: Defines, Constants, and Global Memory

```c
#define NSAMP    512
#define PULSES   512
#define FS       300e6
#define FC       30e9
#define C        3e8
#define PI       3.141592653589793
#define NUM_MC   50
#define NUM_SNR  16

#define FIFO_SIZE_SAMPLES 32768
#define NUM_BATCHES       16
#define SAMPLES_PER_PULSE (2*NSAMP)
#define PULSES_PER_BATCH  (FIFO_SIZE_SAMPLES / SAMPLES_PER_PULSE)

typedef double complex cfloat;

double ga[NSAMP], gb[NSAMP];    // Golay complementary sequences
double txA[2*NSAMP];            // Zero-padded Tx buffer for sequence A
double txB[2*NSAMP];            // Zero-padded Tx buffer for sequence B
cfloat rx[2*NSAMP];             // Received signal per pulse
cfloat cube[NSAMP][PULSES];     // Radar data cube
cfloat RX[2048], CORR[2048];    // FFT working buffers (2048-point)
cfloat TXA[2048], TXB[2048];    // Frequency-domain Tx references
cfloat range_profile[NSAMP];    // Output of one correlation
cfloat doppler_vec[PULSES];     // Temp buffer for Doppler FFT
double r_axis[NSAMP];           // Physical range per bin
double v_axis[PULSES];          // Physical velocity per bin

double snr_list[NUM_SNR] =
    {-45,-44,-43,-42,-41,-40,-39,-38,-37,-36,-35,-34,-33,-32,-31,-30};
```

Listing 15: Golay unoptimized – defines and global memory

Unlike the ZC implementation, the Golay radar uses *real-valued* transmit buffers `txA` and `txB` (type `double`) because the sequences consist only of ± 1 values. The 2048-point FFT buffers `RX` and `CORR` are larger than the 1024-sample pulse to prevent circular convolution wrap-around during matched filtering. Pre-computed axis arrays `r_axis` and `v_axis` convert bin indices to metres and m/s.

6.3.2 G-U Part 2: Gaussian RNG and Signal Power

```c
// randn(): same Box-Muller as ZC (see ZC-U Part 2)

double signal_power(cfloat *x, int N)
{
    double p = 0;
    for (int i = 0; i < N; i++)
        p += creal(x[i])*creal(x[i]) + cimag(x[i])*cimag(x[i]);
    return p / N;
}
```

Listing 16: Golay unoptimized – RNG and signal power

The Golay implementation uses signal-dependent noise scaling: the AWGN power is set relative to the actual received signal power, requiring the `signal_power()` helper to compute $P_s = \frac{1}{N}\sum_i |x[i]|^2$. This is in contrast to the ZC version which uses a fixed `noise_std` derived from SNR directly, because the ZC signal has constant amplitude ($|x[n]| = 1$) and therefore constant power by design.

6.3.3 G-U Part 3: FFT and IFFT

```c
void fft(cfloat *x, int N)  // Works for any power-of-2 N (512 or 2048)
{
    // Bit-reversal permutation
    int i, j = 0;
    for (i = 1; i < N; i++) {
        int bit = N >> 1;
        while (j & bit) { j ^= bit; bit >>= 1; }
        j |= bit;
        if (i < j) { cfloat tmp = x[i]; x[i] = x[j]; x[j] = tmp; }
    }

    // Butterfly stages: twiddle recomputed via cos/sin every iteration
    for (int step = 1; step < N; step *= 2) {
        int jump = step * 2;
        for (int m = 0; m < step; m++) {
            double angle = -PI * m / step;
            cfloat w = cos(angle) + I*sin(angle);  // expensive trig
                per call
            for (i = m; i < N; i += jump) {
                int k = i + step;
                cfloat t = w * x[k];
                cfloat u = x[i];
                x[i] = u + t;
                x[k] = u - t;
```

```
            }
        }
    }
}

void ifft(cfloat *x, int N)
{
    for (int i = 0; i < N; i++) x[i] = conj(x[i]);
    fft(x, N);
    for (int i = 0; i < N; i++) x[i] = conj(x[i]) / N;
}
```

Listing 17: Golay unoptimized – general-length FFT

The Golay FFT is a general-length variant accepting any power-of-2 `N`, used for both 512-point (Doppler) and 2048-point (range correlation) transforms. The twiddle factor is computed as `cos(angle) + I*sin(angle)` for every `m` in every stage on every call. Because the 2048-point correlation FFT is called once per pulse and there are $512 \times 50 \times 16 = 409{,}600$ pulses total, this amounts to millions of redundant trigonometric evaluations.

6.3.4 G-U Part 4: Golay Sequence Generation

```
void generate_golay()
{
    int len = 2;
    ga[0] = 1;  ga[1] =  1;   // A_1 = [1, 1]
    gb[0] = 1;  gb[1] = -1;   // B_1 = [1,-1]

    while (len < NSAMP) {
        double ga_old[NSAMP], gb_old[NSAMP];
        for (int i = 0; i < len; i++) { ga_old[i]=ga[i]; gb_old[i]=gb[i
            ]; }

        for (int i = 0; i < len; i++) {
            ga[i]       =  ga_old[i];    // A_n = [A_{n-1},  B_{n-1}]
            ga[len + i] =  gb_old[i];
            gb[i]       =  ga_old[i];    // B_n = [A_{n-1}, -B_{n-1}]
            gb[len + i] = -gb_old[i];
        }
        len *= 2;
    }
}
```

Listing 18: Golay unoptimized – recursive sequence generation

Starting from $A_1 = [1, 1]$ and $B_1 = [1, -1]$, each iteration doubles the sequence length using the recurrences $A_n = [A_{n-1}, B_{n-1}]$ and $B_n = [A_{n-1}, -B_{n-1}]$. After 8 doublings the length reaches 512. Snapshots into `ga_old`/`gb_old` prevent in-place aliasing. This function is called exactly once before the main processing loop.

6.3.5 G-U Part 5: Transmit Buffer Setup and FFT Pre-computation

```
// Copy Golay sequences with zero padding to length 2*NSAMP
for (int i = 0; i < NSAMP; i++)  { txA[i] = ga[i]; txB[i] = gb[i]; }
```

```
for (int i = NSAMP; i < 2*NSAMP; i++) { txA[i] = 0; txB[i] = 0; }

void prepare_golay_fft()
{
    for (int i = 0; i < 2048; i++) { TXA[i] = 0; TXB[i] = 0; }
    for (int i = 0; i < 2*NSAMP; i++) { TXA[i] = txA[i]; TXB[i] = txB[i
        ]; }

    fft(TXA, 2048);   // Pre-compute frequency-domain version of Tx A
    fft(TXB, 2048);   // Pre-compute frequency-domain version of Tx B
    // Conjugate NOT stored -- computed on the fly inside corr_fft()
}
```

Listing 19: Golay unoptimized – Tx setup and reference FFT

Each 512-sample Golay sequence is first zero-padded to 1024 (to avoid circular wrap-around during correlation), then embedded into a 2048-sample buffer for the FFT. The 2048-point FFTs of both sequences are computed once at startup and stored in `TXA` and `TXB`. However, the conjugate needed for matched filtering is *not* stored — it is computed via `conj(TXA[i])` inside `corr_fft()` on every call.

6.3.6 G-U Part 6: Channel Simulation

```
// Alternate transmission of sequence A and B
double *tx = (pulse % 2 == 0) ? txA : txB;

// Propagation delay as integer sample shift
for (int i = 0; i < 2*NSAMP; i++) rx[i] = 0;
for (int i = 0; i < 2*NSAMP - delay; i++) rx[i + delay] = tx[i];

// Doppler phase rotation
cfloat doppler = cos(2*PI*fd*t_now) + I*sin(2*PI*fd*t_now);
for (int i = 0; i < 2*NSAMP; i++) rx[i] *= doppler;

// AWGN -- noise power proportional to signal power
double Ps    = signal_power(rx, 2*NSAMP);
double Pn    = Ps / snr_lin;
double sigma = sqrt(Pn / 2.0);    // runtime division + sqrt
for (int i = 0; i < 2*NSAMP; i++)
    rx[i] += sigma * (randn() + I*randn());
```

Listing 20: Golay unoptimized – channel simulation per pulse

On even pulses sequence A is transmitted; on odd pulses sequence B. The delay is applied as an integer-sample shift by writing into `rx[delay]`. The Doppler phasor is then applied as a scalar multiplication across all samples. Noise is scaled to match the SNR: the instantaneous signal power `Ps` is measured, divided by the linear SNR to get noise power, and the standard deviation `sigma` is derived accordingly. The `sqrt(Pn / 2.0)` call occurs every pulse.

6.3.7 G-U Part 7: Range Compression

```
void corr_fft(cfloat *rx, int useA)
{
    // Zero-pad received signal to 2048
```

```
    for (int i = 0; i < 2048; i++) RX[i] = 0;
    for (int i = 0; i < 2*NSAMP; i++) RX[i] = rx[i];

    fft(RX, 2048);   // FFT of received signal

    // Frequency-domain matched filtering: Rx * conj(Tx)
    for (int i = 0; i < 2048; i++) {
        if (useA) CORR[i] = RX[i] * conj(TXA[i]);  // conj() every
            sample
        else      CORR[i] = RX[i] * conj(TXB[i]);
    }

    ifft(CORR, 2048);   // IFFT -> time-domain range profile

    for (int r = 0; r < NSAMP; r++) range_profile[r] = CORR[r];
}

// In main loop: alternate which sequence is used
if (pulse % 2 == 0) corr_fft(rx, 1);   // sequence A
else                corr_fft(rx, 0);   // sequence B
```

Listing 21: Golay unoptimized – FFT-based correlation

The received signal is zero-padded to 2048 and FFT'd. The matched filter multiplies element-wise by the conjugate of the stored reference FFT. The `if/else` branch that selects between A and B sits *inside* the 2048-iteration loop, adding a conditional check per sample. The IFFT produces the time-domain range profile; the first 512 samples (the main lobe region) are stored into `range_profile[]`. Summing the A and B correlation results across alternating pulses (done implicitly as both are stored in the same cube) delivers the sidelobe-free combined response.

6.3.8 G-U Part 8: Doppler Processing and Peak Detection

```
// Doppler FFT: copy each row into temp buffer, FFT, shift, write back
for (int r = 0; r < NSAMP; r++) {
    for (int p = 0; p < PULSES; p++)
        doppler_vec[p] = cube[r][p];   // copy-in

    fft(doppler_vec, PULSES);

    for (int p = 0; p < PULSES/2; p++) {  // fftshift
        cfloat tmp           = doppler_vec[p];
        doppler_vec[p]       = doppler_vec[p + PULSES/2];
        doppler_vec[p+PULSES/2] = tmp;
    }
    for (int p = 0; p < PULSES; p++)
        cube[r][p] = doppler_vec[p];   // copy-out
}

// Peak detection using cabs() (includes sqrt)
double max_val = 0;
int r_idx = 0, v_idx = 0;
for (int r = 0; r < NSAMP; r++)
    for (int v = 0; v < PULSES; v++) {
        double mag = cabs(cube[r][v]);   // sqrt(re^2 + im^2)
        if (mag > max_val) { max_val = mag; r_idx = r; v_idx = v; }
```

```
    }

double est_r = r_axis[r_idx];
double est_v = v_axis[v_idx];
```

Listing 22: Golay unoptimized – Doppler FFT and peak detection

The Doppler FFT copies each of the 512 range-bin rows into `doppler_vec`, applies a 512-point FFT, performs an in-place fftshift, and writes the result back — three memory passes per row. Peak detection calls `cabs()` for all 512×512 cells, each involving a `sqrt()`. The estimated range and velocity are read directly from the pre-built `r_axis` and `v_axis` arrays.

6.3.9 G-U Part 9: RMSE and Timing

```
err_r += (est_r - true_r) * (est_r - true_r);
err_v += (est_v - true_v) * (est_v - true_v);

double rmse_r = sqrt(err_r / NUM_MC);
double rmse_v = sqrt(err_v / NUM_MC);

// Timing: XTime_GetTime called at start and end of main()
XTime start, end;
XTime_GetTime(&start);
// ... processing ...
XTime_GetTime(&end);
float exec_time = (float)(end - start) / COUNTS_PER_SECOND;
xil_printf("Execution Time = %f seconds\n\r", exec_time);
```

Listing 23: Golay unoptimized – RMSE and execution time

Unlike the ZC unoptimized code, the Golay unoptimized version *does* include the `XTime` measurement, allowing direct timing comparison against the optimized variant. Squared errors are accumulated over 50 Monte Carlo runs and the RMSE is computed once per SNR point.

6.4 Golay Radar – Optimized

6.4.1 G-O Part 1: New Defines and Memory

```
#define FFT_RANGE 2048        // Named constant for correlation FFT size
#define INV_SQRT2 0.70710678 // Compile-time reciprocal of sqrt(2)

/* Pre-stored conjugates of transmit FFTs */
cfloat TXA_CONJ[FFT_RANGE];
cfloat TXB_CONJ[FFT_RANGE];

/* Full twiddle table for 2048-point FFT */
cfloat twiddle[FFT_RANGE];

/* Cached scale factors -- computed once, reused every bin/pulse */
double range_scale = C / (2 * FS);    // = 0.5 m/bin
double vel_scale   = lambda / 2;      // converts fd to velocity
```

Listing 24: Golay optimized – additional defines and memory

Four optimizations are introduced at the data-layout level. `TXA_CONJ` and `TXB_CONJ` store the element-wise conjugates of the reference FFTs, replacing runtime `conj()` calls in the inner correlation loop. `twiddle[]` is a 2048-element table of all twiddle factors, replacing the runtime `cos/sin` inside the FFT butterfly. `range_scale` and `vel_scale` cache frequently used division results as scalars, computed once at startup.

6.4.2 G-O Part 2: Twiddle Table and Optimized FFT

```c
void init_twiddle()
{
    for (int i = 0; i < FFT_RANGE; i++) {
        double angle = -2 * PI * i / FFT_RANGE;
        twiddle[i] = cos(angle) + I*sin(angle);   // computed ONCE
    }
}

void fft(cfloat *x, int N)
{
    // Bit-reversal (unchanged)
    // ...

    for (int step = 1; step < N; step *= 2) {
        int jump    = step * 2;
        int tw_step = FFT_RANGE / jump;  // stride into full 2048 table

        for (int m = 0; m < step; m++) {
            cfloat w = twiddle[m * tw_step];  // array load, no trig

            for (int i = m; i < N; i += jump) {
                int k = i + step;
                cfloat t = w * x[k];
                cfloat u = x[i];
                x[i] = u + t;
                x[k] = u - t;
            }
        }
    }
}
```

Listing 25: Golay optimized – twiddle init and FFT with table lookup

`init_twiddle()` builds a 2048-entry table of all twiddle factors once. For any sub-transform of length $N \leq 2048$, the twiddle factor for butterfly index m at stage size `jump` is located at `twiddle[m * (FFT_RANGE/jump)]`. This stride relationship ensures correct twiddle selection without any additional computation. Each `cos/sin` call is replaced by a 16-byte memory load.

6.4.3 G-O Part 3: Precomputed Conjugate References

```c
void prepare_golay_fft()
{
    // ... (same zero-padding and FFT as unoptimized) ...
    fft(TXA, FFT_RANGE);
    fft(TXB, FFT_RANGE);

```

```
    // Store conjugates ONCE -- replaces conj() in inner loop
    for (int i = 0; i < FFT_RANGE; i++) {
        TXA_CONJ[i] = conj(TXA[i]);
        TXB_CONJ[i] = conj(TXB[i]);
    }
}
```

Listing 26: Golay optimized – conjugates stored after prepare_golay_fft

After computing the 2048-point FFTs of both Golay sequences, their conjugates are immediately stored in `TXA_CONJ` and `TXB_CONJ`. This single extra pass (4096 `conj()` calls at startup) eliminates 4096 `conj()` calls *per pulse per trial per SNR*, saving roughly 210 million function calls over the full run.

6.4.4 G-O Part 4: Optimized Correlation

```
void corr_fft(cfloat *rx, int useA)
{
    for (int i = 0; i < FFT_RANGE; i++) RX[i] = 0;
    for (int i = 0; i < 2*NSAMP; i++) RX[i] = rx[i];

    fft(RX, FFT_RANGE);

    // Branch moved OUTSIDE the loop; direct multiply from table
    if (useA) {
        for (int i = 0; i < FFT_RANGE; i++)
            CORR[i] = RX[i] * TXA_CONJ[i];   // array load, no conj()
    } else {
        for (int i = 0; i < FFT_RANGE; i++)
            CORR[i] = RX[i] * TXB_CONJ[i];
    }

    ifft(CORR, FFT_RANGE);
    for (int r = 0; r < NSAMP; r++) range_profile[r] = CORR[r];
}
```

Listing 27: Golay optimized – correlation with precomputed conjugates

Two changes versus the unoptimized version: (1) the `if/else` branch that selects sequence A or B is moved *outside* the 2048-iteration loop, eliminating a conditional check on every element; (2) `conj(TXA[i])` is replaced by a direct load from `TXA_CONJ[i]`. The mathematical result is identical: $Y[k] = R[k] \cdot \overline{T[k]}$.

6.4.5 G-O Part 5: AWGN Optimisation

```
/* Unoptimized: */
double sigma = sqrt(Pn / 2.0);

/* Optimized: */
double sigma = sqrt(Pn) * INV_SQRT2;
```

Listing 28: Golay optimized – INV_SQRT2 replaces runtime division

`sqrt(Pn / 2.0)` performs a floating-point division followed by a square root. The optimized form `sqrt(Pn) * INV_SQRT2` replaces the division with a multiply by the compile-time constant $1/\sqrt{2}$, which is typically a single-cycle FPU instruction. The result is numerically identical.

6.4.6 G-O Part 6: Doppler FFT – Direct In-Place on Cube

```
for (int r = 0; r < NSAMP; r++) {
    fft(cube[r], PULSES);   // pass pointer directly to cube row

    for (int p = 0; p < PULSES/2; p++) {   // in-place fftshift
        cfloat tmp            = cube[r][p];
        cube[r][p]            = cube[r][p + PULSES/2];
        cube[r][p + PULSES/2] = tmp;
    }
    // No write-back needed -- cube[r] was modified in-place
}
```

Listing 29: Golay optimized – Doppler FFT directly on cube row

The unoptimized version used a three-step sequence: copy row to `doppler_vec`, FFT on `doppler_vec`, copy back to `cube[r]`. The optimized version passes `cube[r]` — a pointer to a contiguous 512-element block in the data cube — directly to `fft()`. Because the FFT operates in-place, no copies are needed, saving $2 \times 512 \times 512 = 524{,}288$ complex assignments per Monte Carlo run.

6.4.7 G-O Part 7: Peak Detection – Squared Magnitude

```
double max_val = 0;
int r_idx = 0, v_idx = 0;

for (int r = 0; r < NSAMP; r++)
    for (int v = 0; v < PULSES; v++) {
        double re  = creal(cube[r][v]);
        double im  = cimag(cube[r][v]);
        double mag = re*re + im*im;    // squared magnitude -- no sqrt
            ()
        if (mag > max_val) { max_val = mag; r_idx = r; v_idx = v; }
    }

double est_r = r_idx * range_scale;   // cached scalar multiply
double est_v = v_axis[v_idx];
```

Listing 30: Golay optimized – peak detection without sqrt

Since $f(x) = x^2$ is monotonically increasing for $x \geq 0$, comparing squared magnitudes is equivalent to comparing magnitudes for the purpose of finding the maximum. This eliminates a `sqrt()` call for all $512 \times 512 = 262{,}144$ cells. Additionally, the range estimate uses the pre-cached scalar `range_scale` instead of recomputing `C / (2*FS)` each time.

7 Performance Evaluation and Outputs

7.1 Monte Carlo Simulation Framework

To statistically validate system reliability, Monte Carlo simulations were executed 50 times for each SNR value. Random target ranges and velocities were independently generated for each run, and the SNR was swept over 16 values from $-45\,\text{dB}$ to $-30\,\text{dB}$.

7.2 RMSE Metrics

System performance is quantified using Root Mean Square Error (RMSE) for both range and velocity:

$$\mathrm{RMSE}_R = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(R_{\text{true}}^{(i)} - R_{\text{estimated}}^{(i)}\right)^2} \tag{27}$$

$$\mathrm{RMSE}_V = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(V_{\text{true}}^{(i)} - V_{\text{estimated}}^{(i)}\right)^2} \tag{28}$$

where $N = 50$ is the total number of Monte Carlo trials. Lower RMSE values indicate higher estimation accuracy.

7.3 Golay Radar Performance

Evaluated at a PRI of 1024 with an SNR of $-30\,\mathrm{dB}$, the system produced a clear Range-Doppler map successfully pinpointing the target. At extremely low SNRs (e.g., $-40\,\mathrm{dB}$), both RMSE_R and RMSE_V are high. However, the RMSE drops sharply and stabilises to near-ideal accuracy as the SNR crosses approximately $-38\,\mathrm{dB}$, validating the robustness of the sidelobe-free complementary architecture.

7.4 ZC Radar Performance

Under identical severe noise conditions (SNR $-30\,\mathrm{dB}$), the ZC sequences also produced an accurate 2D Delay-Doppler map. The range and velocity accuracy versus SNR plots mirror the Golay behaviour: the RMSE decreases sharply between $-40\,\mathrm{dB}$ and $-38\,\mathrm{dB}$, maintaining strong stability at higher SNR values. This confirms the effectiveness of the CAZAC property in practical noisy environments.